

정보 입력 부문에서 `except` 문에서 `return []`을 사용하여 만약 잘못된 입력으로 기존의 데이터가 오염되면 빈 리스트를 반환함으로써 입력된 정보들을 새로 초기화 하는 역할을 할 수 있는거야?

정확합니다! 코드의 흐름과 의도를 완벽하게 꿰뚫어 보셨네요.

말씀하신 대로 `except` 문에서 `return []` 을 반환하는 것은 "입력 도중 발생한 오염된 데이터를 버리고 상태를 초기화하여, 프로그램이 꼬이는 것을 막는 방패" 역할을 합니다.

구체적으로 프로그램 안에서 어떤 원리로 이 방어망이 작동하는지 단계별로 설명해 드릴게요.

빈 리스트(`[]`)가 데이터를 보호하는 3단계 원리

1단계: 반쪽짜리(오염된) 데이터 방지

만약 사용자가 기온, 예상 기온까지는 숫자로 잘 입력했는데 습도에서 실수로 "오십"이라고 문자를 쳤다고 가정해 보겠습니다.

이 순간 즉시 에러가 발생하면서 코드는 `try` 블록을 탈출해 `except` 로 넘어갑니다. 이때 `return []` 을 해주지 않으면, 습도나 풍속 데이터가 아예 없거나 엉뚱한 값이 들어간 '반쪽짜리 쓰레기 데이터'가 저장될 위험이 있습니다.

2단계: 메인 메뉴의 데이터 덮어쓰기 (초기화)

메인 메뉴의 코드를 보면 다음과 같이 되어 있습니다.

2단계: 메인 메뉴의 데이터 덮어쓰기 (초기화)

메인 메뉴의 코드를 보면 다음과 같이 되어 있습니다.

Python

```
if choice == '1':
    weather_data = input_weather_info() # 실패 시 weather_data에 []가 들어감
    if weather_data: # 빈 리스트([])는 False로 취급됨
        print("\n정보가 저장되었습니다.")
```

`except` 에서 던져준 빈 리스트(`[]`)를 `weather_data` 가 그대로 받습니다. 즉, 기존에 입력하던 찌꺼기 데이터가 남지 않고 완전한 백지상태(초기화)가 되는 것입니다. 빈 리스트이기 때문에 "정보가 저장되었습니다."라는 메시지도 출력되지 않아, 사용자는 입력에 실패했음을 직관적으로 알 수 있습니다.

3단계: 다른 메뉴(2~5번) 접근 차단

가장 중요한 역할입니다. `check_data(data)` 함수는 리스트가 비어있는지(`if not data:`)를 가장 먼저 검사합니다.

입력 실패로 인해 `weather_data` 가 `[]` 로 초기화되었기 때문에, 사용자가 2, 3, 4, 5번 메뉴를 누르더라도 "먼저 1번 메뉴에서 정보를 올바르게 입력해주세요!"라며 안전하게 접근을 막아줍니다.

결론적으로,

`return []` 은 단순히 에러를 무시하는 것을 넘어서서 ① 찌꺼기 데이터 삭제, ② 변수 초기화, ③ 타 메뉴 접근 차단이라는 세 가지 핵심 안전장치를 한 번에 걸어주는 아주 똑똑한 코드입니다. 이 부분의 역할을 정확히 파악하신 걸 보니 프로그래밍의 '데이터 흐름(Data Flow)'을 읽는 눈이 아주 좋습니다!